

Adaptive Niching-Based Gradient-Accelerated Differential Evolution for High-Dimensional Nonconvex Optimization

Qi Yu[✉], Xijun Liang[✉], *Member, IEEE*, Jinneng Liu, Pu Tian, and Ling Jian[✉], *Senior Member, IEEE*

Abstract—Nonconvex optimization presents significant challenges in many fields, particularly in training deep neural networks (DNNs), where poor local minima can degrade generalization—especially with limited data. High-dimensional nonconvex optimization presents two central challenges: 1) effectively balancing global exploration with rapid local exploitation and 2) establishing convergence guarantees, particularly with sparse individuals under nonsmooth regularizations. To address these limitations, we propose adaptive niching-based gradient-accelerated DE (AdaptiveGDE), an AdaptiveGDE differential evolution (DE) algorithm. It introduces a novel two-step mutation operator that decouples differential mutation and gradient descent, allowing independent control of exploration and exploitation. An adaptive niching strategy dynamically adjusts the number of subpopulations based on population similarity and iteration progress, enabling diverse early exploration and refined late-stage convergence. Under relaxed smoothness assumptions and approximate ℓ_1 regularization, we provide convergence guarantees in expectation to a near-optimal solution within $\mathcal{O}(1/\epsilon^4)$ iterations. Extensive experiments show that AdaptiveGDE achieves robust global exploration on complex multimodal functions, strong local exploitation on convex problems, and significantly improves test accuracy and loss in DNN training, especially under limited data scenarios.

Index Terms—Deep neural network (DNN), differential evolution (DE), gradient descent, nonconvex optimization.

I. INTRODUCTION

NONCONVEX optimization poses a fundamental challenge in numerous fields. For example, the optimization of high-dimensional multimodal functions can exhibit rapid variations and contain numerous local minima across a vast search space. Similarly, training deep neural networks (DNNs)

involves solving highly complex, high-dimensional nonconvex optimization problems. For example, the architectures used in our experiments, such as ResNet-18 and ResNet-50, contain approximately 11.7 and 25.6 million parameters, respectively, with each parameter representing a distinct dimension in the underlying search space.

In particular, when training data is limited, optimization algorithms are more likely to become trapped in poor local minima, leading to instability and degraded model performance. One of the central challenges in high-dimensional nonconvex optimization lies in designing algorithms that can simultaneously achieve two inherently conflicting objectives: strong global exploration to escape poor local minima and fast local exploitation to ensure efficient convergence near promising regions.

Gradient descent and stochastic gradient descent (SGD) are widely used for nonconvex optimization due to their computational efficiency and convergence guarantees. Under standard assumptions, these methods typically converge to approximate first-order stationary points at rates that are dimension-independent, making them effective for high-dimensional problems. To better handle the complex loss surfaces of DNNs, numerous SGD variants have been proposed. For example, Adam [1] adapts learning rates using first- and second-order gradient moments, promoting faster and more stable convergence. Recent methods further combine Adam's adaptability with Nesterov acceleration [2] to enhance convergence speed and robustness. Notable examples include AdaBelief [3], NAdam [4], Win [5], Lookahead [6], Lion [7], and Adan [8].

Despite their advantages, these gradient-based methods rely solely on local gradient information, making them more prone to getting trapped in suboptimal regions—particularly in complex, high-dimensional landscapes.

In contrast, evolutionary algorithms (EAs) like differential evolution (DE) excel in nonconvex optimization by maintaining population diversity and performing robust global search, helping avoid suboptimal local optima. This makes DE a valuable tool in computational intelligence, including DNN training [9], [10], [11]. However, DE struggles in high-dimensional settings due to its lack of gradient use, which hampers local search efficiency. The curse of dimensionality reduces sampling efficiency and makes exploration–exploitation trade-offs intractable.

Gradient-based optimizers and EAs offer complementary strengths in nonconvex optimization, inspiring hybrid approaches that combine their respective advantages. The

Received 10 June 2025; revised 29 September 2025 and 17 December 2025; accepted 2 March 2026. Date of publication 17 March 2026; date of current version 2 September 2026. This work was supported in part by the National Key Research and Development Program of China under Grant 2021YFA1000102, in part by the Key Program of Joint Fund of the National Natural Science Foundation of China and Shandong Province under Grant U22A20586, in part by Shandong Provincial Natural Science Foundation under Grant ZR2023MF002, and in part by the Fundamental Research Funds for the Central Universities under Grant 24CX03004A. (Corresponding author: Xijun Liang.)

Qi Yu, Xijun Liang, Jinneng Liu, and Pu Tian are with the College of Science, China University of Petroleum, Qingdao, Shandong 266580, China (e-mail: b23090007@s.upc.edu.cn; liangxijunsd@upc.edu.cn; s23090032@s.upc.edu.cn; b24090007@s.upc.edu.cn).

Ling Jian is with the School of Economics and Management, China University of Petroleum, Qingdao, Shandong 266580, China (e-mail: bebetter@upc.edu.cn).

This article has supplementary downloadable material available at <https://doi.org/10.1109/TNNLS.2026.3671634>, provided by the authors.

Digital Object Identifier 10.1109/TNNLS.2026.3671634

core of such hybrid methods lies in integrating the global exploration capacity inherent in EAs with the local convergence efficiency of gradient-based algorithms. One approach alternates between the two: for example, ESGD [12] interleaves SGD and evolutionary updates during DNN training, while HGDSA [13] employs a population-based global search (MSaGDSA) followed by backpropagation for local optimization. Another approach incorporates gradient information directly into EAs, as in GEMONN [14], which guides mutation by gradient signals, and NGDE [15], which integrates gradient directions into individual update steps.

Although hybrid methods have advanced nonconvex optimization, they each face limitations in addressing the two key challenges inherent to high-dimensional nonconvex problems.

- 1) **Balancing effective global exploration and fast local exploitation.** ESGD [12] employs coordinate-based exploration strategies but neglects gradient information, resulting in slow convergence during the exploration phase. GEMONN [14] implicitly incorporates gradients in its mutation process, which may hinder exploitation efficiency. Meanwhile, NGDE [15] tightly integrates global and local search directions, potentially reducing flexibility in the exploration process.
- 2) **Managing storage overhead and the need for sparsity.** Hybrid methods often incur substantial storage costs and inherently benefit from sparse solutions. However, there is a notable lack of theoretical analysis regarding the convergence properties of iteratively updated individuals with sparse parameterizations.

Motivated by these limitations, this article proposes the adaptive niching gradient-accelerated DE algorithm. It begins by generating a population of feasible solutions, which are adaptively divided into ecological niches to explore diverse regions. Differential mutation drives early exploration: high-fitness individuals mutate based on their niche's best, while low-fitness ones mutate randomly. Gradient descent is then applied at intermediate points to speed up convergence. The number of niches is adjusted based on individual similarity, and a two-step gradient-accelerated mutation operator ensures early-stage exploration and late-stage exploitation. The main contributions are as follows.

- 1) *Gradient-Accelerated Mutation Operator:* We propose a novel operator that decouples differential mutation from gradient descent. Instead of directly combining mutation and gradient directions, the operator first explores using differential mutation and then refines intermediate solutions via gradient-based acceleration. This decoupling enables independent tuning of exploration and exploitation, offering greater flexibility.
- 2) *Adaptive Niching Strategy and AdaptiveGDE Algorithm:* To enhance the effectiveness of the mutation process, we propose an adaptive niching strategy that dynamically adjusts the number of niches according to individual similarity and iteration progress. This mechanism encourages broad exploration during early stages and focused exploitation in later stages, operating in synergy with the designed mutation operator. Building upon this

strategy and the gradient-based mutation, we develop the adaptive niching gradient-accelerated DE algorithm.

- 3) *Theoretical Guarantee:* Hybrid methods typically lack rigorous convergence results due to: 1) the stochasticity of evolutionary operators and 2) the nonsmoothness introduced by sparsity-inducing regularizers. To overcome these challenges, we propose a two-stage mutation mechanism that decouples global exploration from local refinement. By integrating an approximate ℓ_1 regularizer with relaxed smoothness assumptions, we make the convergence analysis tractable despite nonsmoothness and randomness. We prove that adaptive niching-based gradient-accelerated DE (AdaptiveGDE) converges in expectation to an ϵ -approximate stationary point within $\mathcal{O}(1/\epsilon^4)$ iterations, without requiring local strong convexity or Lipschitz gradient continuity. These results provide a solid theoretical foundation for gradient-accelerated evolutionary frameworks on sparsity-regularized individuals.

AdaptiveGDE is empirically validated on two fronts: high-dimensional multimodal function minimization and DNN training. On benchmark multimodal functions with up to 50 000 dimensions, it demonstrates significant improvements in exploration and exploitation, achieving better optimized function values. In DNN training with limited data, AdaptiveGDE consistently enhances test accuracy while substantially reducing both test and validation losses. For instance, with limited training samples, it improves LeNet-5 accuracy by 1.22% and reduces test loss by 36.05%; on ResNet-18, it achieves accuracy gains of 1.64% on MNIST and FashionMNIST, and 5.06% on CIFAR-10, CIFAR-100, and SVHN.

The structure of this article is organized as follows. Section II reviews the background and related work, focusing on gradient-based optimization algorithms, niching-based DE algorithms, and hybrid EAs. Section III introduces the proposed AdaptiveGDE method, and Section IV presents its convergence analysis. Section V reports the experimental results, and Section VI concludes this article.

II. RELATED WORKS

A. Gradient-Based Algorithms

Gradient descent algorithms are foundational in classical nonconvex optimization, offering convergence rates independent of the problem's dimensionality. They include SGD and its accelerated variants, which are widely used in DNN optimization. SGD approximates the full gradient using mini-batches, reducing computational cost, while momentum-based methods like classical momentum (CM) [16] and Nesterov accelerated gradient (NAG) [2] improve convergence by mitigating oscillations and speeding up iterations, with NAG theoretically faster for convex problems. Despite their success, methods like NAG face limitations in training DNNs. Adam [1], a widely adopted optimizer, adaptively adjusts learning rates based on historical gradients, inspiring variants like Adabound [17] and Adabelief [3] for enhanced stability and accuracy. AdamP [18] prevents gradient collapse with

adaptive shearing, Radam [19] minimizes gradient standard deviation for stability, and AdamW [20] separates primary objectives from regularization to improve generalization across applications. Lookahead [6] maintains fast and slow weight copies, periodically synchronizing them to stabilize training and enhance generalization. Lion [7] employs sign-based updates with momentum-like direction accumulation, showing competitive performance in large-scale deep learning tasks.

Recent studies integrate Nesterov acceleration to improve DNN training. NAdam [4] simplifies Nesterov acceleration by estimating the gradient's first moments in Adam, but does not utilize gradients at extrapolation points. Zhou et al. [5] proposed weight-decay-integrated Nesterov acceleration (Win), which integrates weight decay and Nesterov strategies to accelerate adaptive gradient methods, improving DNN training efficiency. Xie et al. [8] introduced Adan, the adaptive Nesterov momentum algorithm, which refines Nesterov momentum estimation without requiring gradient computation at extrapolation points.

B. Niching-Based DE Algorithms

EAs, inspired by biological evolution, optimize solutions iteratively through global search and selection [21]. Among these, evolution strategies (ESs), notably CMA-ES, and DE are prominent variants for continuous nonconvex optimization; DE is more commonly applied to neural network training [9], [10], [22]. To improve the ability of EAs to solve complex optimization problems, researchers have introduced niching methods to promote diversity and explore multiple optimal solutions. Traditional methods like crowding, speciation, and fitness sharing rely on parameters such as crowding factor and niche radius, making them sensitive to parameter settings [23]. To address this, dynamic niching methods have been proposed. Qu et al. [24] introduced a neighborhood mutation strategy for niche DE algorithms, while Yang et al. [25] developed a dynamic cluster size approach to balance exploration and exploitation, reducing sensitivity to cluster size parameters.

The niching DE algorithm, combining niching methods with evolutionary strategies, effectively addresses complex multimodal optimization problems. Wang et al. [26] developed an automated niche technique using affine propagation clustering for multimodal optimization. Liang et al. [27] introduced a co-EA with a miniature context-based strategy to enhance local search. Sheng et al. [28] proposed an adaptive miniature context DE algorithm incorporating k-means for partitioned data clustering. Yang et al. [29] designed the triple competitive DE (TCDE) algorithm, which introduces a triple competition mechanism. The individuals are randomly grouped into independent triples, fostering competition within each group.

C. EAs Hybridizing Gradient-Based Search

Fusing EAs with gradient-based methods combines the global search strength of EAs with the fast convergence of gradient approaches, enhancing optimization performance. This hybrid strategy effectively addresses complex nonconvex problems, particularly in neural network optimization.

Jin et al. [30] proposed perturbed gradient descent (PGD), which integrates the mutation operator from EAs into gradient descent to avoid saddle points and bad local optima. Cui et al. [12] introduced the evolutionary SGD (ESGD) algorithm for DNN optimization, alternating between SGD and EA steps to improve population fitness. Xue et al. [21] introduced evolutionary gradient descent (EGD), which combines EA population and mutation operators with gradient descent. In addition, Xue et al. [13] developed the hybrid gradient descent search algorithm (HGDSA), which begins with an extensive search using gradient descent and later refines the solution with local search via backpropagation. Xue et al. [31] proposed EDEAdam, an ensemble of DE and Adam, evolving two subpopulations in parallel for better global and local search results. Yang et al. [14] developed GEMONN, a gradient-guided evolutionary multiobjective method for training DNNs, using a gradient-based simulated binary crossover (gSBX) operator for offspring generation. Yu et al. [15] proposed the niching-based gradient-directed evolution (NGDE) algorithm for high-dimensional nonconvex optimization, using niche-based exploration and gradient-directed mutation for offspring creation. These algorithms either incorporate gradient direction only indirectly within the mutation operator or directly add the differential mutation direction to the gradient, which limits their exploitation efficiency and slows convergence.

III. PROPOSED METHOD

In this section, we introduce the AdaptiveGDE algorithm, developed to solve the following nonconvex optimization problem:

$$\min_{x \in \mathcal{X}} f(x) \quad (1)$$

where $f : \mathbb{R}^d \rightarrow \mathbb{R}$ is a nonconvex objective function with a computable or estimable gradient. The feasible set $\mathcal{X} \subseteq \mathbb{R}^d$ is assumed to have a simple structure. In this work, we primarily consider the unconstrained case where $\mathcal{X} = \mathbb{R}^d$. The formulation in (1) encompasses a wide range of multimodal optimization and machine learning problems, including the training of DNNs. We focus on both categories, with particular attention to scenarios where f represents an empirical loss over a set of training samples. For clarity and consistency, Table S.1 in the supplementary material summarizes the mathematical notation in Appendix A.

A. Overall Framework

Fig. 1 presents the flowchart of AdaptiveGDE. The algorithm begins by randomly initializing a population of μ individuals within the feasible region $\mathcal{X}$. These individuals are then grouped using an adaptive niching method. Subsequently, individual updates are performed by adaptively selecting a gradient-accelerated differential mutation strategy tailored to each individual's fitness. The core idea behind AdaptiveGDE is to balance global exploration and local exploitation through a gradient-accelerated mutation operator and an adaptive niching mechanism. The mutation operator decouples the gradient descent component from the differential mutation process by applying gradient descent (or multistep SGD) at the trial point

Algorithm 1 AdaptiveGDE Algorithm**Input:** the population size μ , the number of generations K .**Output:** the population P^K and the individual with the highest fitness.

```

1: Initialize the population  $P^0 = \{x_1^0, x_2^0, \dots, x_\mu^0\}$ , set  $k = 0$ .
2: while  $k < K$  do
3:    $P^{k+1} \leftarrow \emptyset$ 
4:   Apply Algorithm 2 to divide the population  $P^k$  into
     several niches  $N_1^k, \dots, N_m^k$ ,  $m$  is the number of the niches
     in the  $k$ -th generation
5:   for  $i = 1 : m$  do
6:     for  $j = 1 : m_i$  do  $m_i$  is the size of niche  $N_i^k$ 
7:       Apply Algorithm 3 or Algorithm S.1 to generate
       the candidate offspring  $x_{i,j}^{k+1}$  based on  $x_{i,j}^k$ 
8:        $P^{k+1} \leftarrow P^{k+1} \cup \{x_{i,j}^{k+1}\}$ 
9:     end for
10:  end for
11:   $k \leftarrow k + 1$ 
12: end while
13: return  $P^K$ 

```

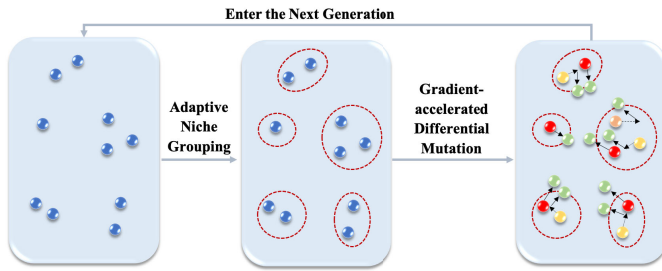


Fig. 1. Illustration of the AdaptiveGDE framework. Red circles denote niches, while colored dots represent individuals with different fitness levels: red (highest), yellow (high), orange (low), and green (offspring).

generated via differential mutation. This decoupling allows the local search along the negative gradient to operate independently of the exploratory behavior driven by differential mutation, enabling both components to be tuned separately throughout the population's evolution. In addition, the adaptive niching method dynamically adjusts the number of niches based on pairwise similarity among individuals. It promotes larger niches in the early stages to enhance global exploration, and encourages smaller niches in later stages to improve local exploitation.

The detailed procedure of the adaptive niching-based gradient-accelerated DE algorithm is outlined in Algorithm 1. At the beginning of each generation k , the population P^k is partitioned into adaptive niches as described in Algorithm 2. For each individual $x_{i,j}^k$ within niche N_i^k , a candidate offspring $x_{i,j}^{k+1}$ is generated via the gradient-accelerated differential mutation operator (see Algorithm 3 or Algorithm S.1 in the Supplementary Material). These offspring collectively form the next generation's population P^{k+1} .

Fig. 2(a)–(c) illustrates the individual search trajectories of MGD, NGDE, and AdaptiveGDE on the Bent Cigar function

$f(\mathbf{x}) = x_1^2 + 100x_2^2$ over $[-10, 10]^2$, using a constant step size and initialized with a common population of three individuals.

- 1) Fig. 2(a) MGD: As a purely gradient-based method, individuals slow down significantly once the x_2 coordinate approaches zero.
- 2) Fig. 2(b) NGDE: Individuals oscillate and then stagnate in a small region near $(1.0, 0)$.
- 3) Fig. 2(c) AdaptiveGDE: Individuals rapidly converge to the global optimum $(0, 0)$.
- 4) Fig. 2(d): It presents the corresponding iterated logarithmic error $\ln(f(\mathbf{x}_{\text{best}}^k) - f^*)$.

Since the function is strongly convex with a unique minimum and a high condition number, these figures emphasize each algorithm's local exploitation ability.

NGDE updates individuals by combining two directional components, the negative gradient and differential mutation, which can lead to conflicting directions and hinder effective exploitation. In contrast, AdaptiveGDE adopts a two-stage mutation mechanism. First, differential mutation along $x_{\text{best}}^k - x^k$ enables broad exploration and generates a trial point. Then, gradient descent refines this point via local exploitation. This decoupled design balances exploration and exploitation more effectively and demonstrates stable and efficient local convergence behavior in this example.

B. Adaptive Niching Method

Many EAs employ niching techniques to preserve population diversity. However, conventional niching methods often rely on fixed parameters—such as a predefined niche radius or a constant number of niches—which limits their adaptability during the dynamic evolutionary process. To better balance global exploration and local exploitation in AdaptiveGDE, we propose an adaptive niching strategy that dynamically adjusts the number of niches based on individual similarity and the current evolutionary stage. In the early epochs, fewer niches with more individuals facilitate broad exploration of the search space. As the algorithm evolves, the number of niches increases while the number of individuals per niche decreases, thereby enhancing local exploitation within each niche. This adaptive mechanism complements the mutation operator described in Section III-C.

The details of this niching method are presented in Algorithm 2, where μ is the population size, m is the number of niches, and $df_{\max}$ and $df_{\min}$ are the distance factor boundaries. The method includes two key components.

- 1) *Similarity Measure*: To overcome the limitations of Euclidean distance in high-dimensional spaces, we adopt cosine similarity, which measures the angular difference between vectors and is more robust in such settings. To ensure compatibility with subsequent operations, the cosine similarity is normalized to the range $[0, 1]$ as follows:

$$ncos = \frac{1}{2} \left(1 + \frac{x_1 \cdot x_2}{|x_1||x_2|} \right) \quad (2)$$

where x_1 and x_2 denote two individuals randomly selected from the population. The mean cosine similarity, $\overline{ncos}$, is then computed across all unique pairs in the

Algorithm 2 Adaptive Niching Method

Input: the population P^k , the population size μ , the distance factor $df_{\max}$ and $df_{\min}$

Output: the niches $N_1^k, \dots, N_m^k$

- 1: Calculate the mean of the cosine similarity $\overline{ncos}$ between all individuals in P^k according to Eq. (3).
- 2: Calculate the distance factor df^k according to Eq. (5).
- 3: Calculate the distance threshold DT^k according to Eq. (4).
- 4: $m \leftarrow 1$ $\triangleright m$: the number of niches
- 5: $N_1^k \leftarrow \{p_1^k\}$ $\triangleright P^k = \{p_1^k, p_2^k, \dots, p_\mu^k\}$
- 6: **for** $i = 2 : \mu$ **do**
- 7: $niche_found \leftarrow \text{False}$ $\triangleright p_i^k$ is not grouped in niches
- 8: **for** $j = 1 : m$ **do**
- 9: Calculate the cosine similarity $ncos_{i,j}^k$ between p_i^k and individuals in niche N_j^k
- 10: **if** $ncos_{i,j}^k < DT^k$ **then**
- 11: $N_j^k \leftarrow N_j^k \cup \{p_i^k\}$
- 12: $niche_found \leftarrow \text{True}$ $\triangleright p_i^k$ is grouped in N_j^k
- 13: **break**
- 14: **end if**
- 15: **end for**
- 16: **if** $niche_found$ is False **then**
- 17: $m \leftarrow m + 1$
- 18: $N_m^k \leftarrow \{p_i^k\}$
- 19: $niche_found \leftarrow \text{True}$ $\triangleright p_i^k$ is grouped in N_m^k
- 20: **end if**
- 21: **end for**
- 22: **return** $N^k = \{N_1^k, \dots, N_m^k\}$

population to provide a global measure of population similarity

$$\overline{ncos} = \frac{1}{\binom{n}{2}} \sum_{i=1}^n \sum_{j=i+1}^n ncos(x_i, x_j) \quad (3)$$

where $\binom{n}{2} = (n(n-1))/2$ is the total number of pairwise combinations among the n individuals.

- 2) *Adaptive Distance Threshold:* To enable dynamic niche formation throughout the evolutionary process, we introduce an adaptive similarity threshold that evolves based on both the average population similarity and the iteration count. Individuals with higher similarity are assigned to existing niches, while those with lower similarity initiate new ones. The similarity threshold at iteration k , denoted as DT^k , is defined as

$$DT^k = \overline{ncos} \cdot df^k \quad (4)$$

where $\overline{ncos}$ is the mean cosine similarity [as defined in (3)], and df^k is the distance factor at iteration k , given by

$$df^k = df_{\min} + (df_{\max} - df_{\min}) \cdot \sqrt{\frac{k}{K}} \quad (5)$$

with K representing the total number of iterations. This strategy begins with a low threshold DT^k to facilitate global exploration of the search space, and then gradually increases the threshold DT^k to refine niches and accelerate convergence.

Algorithm 3 Gradient-Accelerated Differential Mutation Using the Full Gradient

Input: the population P^k , the niche N_i^k , the individual $x_{i,j}^k$, the learning rate η_k , the mutation factor $F_{\max}$ and $F_{\min}$, the fitness threshold p_f

Output: the offspring $x_{i,j}^{k+1}$

- 1: Find the individual $x_{i,best}^k$ with the highest fitness value in niche N_i^k .
- 2: Calculate the normalized function value $p_{i,j}^k$ at $x_{i,j}^k$ with Eq.(6).
- 3: Calculate the $F_{i,j}^k$ according to Eq.(12)
- 4: **if** $p_{i,j}^k > p_f$ **then**
- 5: $\bar{x}_{i,j}^k = x_{i,j}^k + F_{i,j}^k \cdot (x_1 - x_2)$ with x_1, x_2 randomly selected from P^k
- 6: **else**
- 7: $\bar{x}_{i,j}^k = x_{i,j}^k + F_{i,j}^k \cdot (x_{i,best}^k - x_{i,j}^k)$
- 8: **end if**
- 9: Calculate $\bar{g}_{i,j}^k$ as $\nabla f(\bar{x}_{i,j}^k)$.
- 10: $x_{i,j}^{k+1} = \bar{x}_{i,j}^k - \eta_k \cdot \bar{g}_{i,j}^k$
- 11: **return** $x_{i,j}^{k+1}$

C. Gradient-Accelerated Differential Mutation Operator

After dividing the population into niches, individuals from different niches collaborate to explore diverse local optima. To balance exploitation and exploration, we adapt mutation operators based on individual fitness and incorporate gradient acceleration to improve convergence speed. In particular, we decouple global exploration via differential mutation from local refinement along the negative gradient. A trial point is first generated by mutating along either the direction toward the best individual in the niche or the vector defined by two randomly selected individuals within the same niche. Subsequently, the individual performs a local search along the negative gradient at the trial point. By separating these two operations, the mutation strategy allows each to be tuned independently, providing a more flexible balance between exploration and exploitation. The detailed mutation procedure is described in Algorithm 3, assuming that the full gradient can be computed efficiently.

Algorithm 3 assigns fast convergence responsibilities to individuals with higher fitness within each niche, while others focus on exploring broader regions. To evaluate fitness, we normalize each individual's function value as follows:

$$p_{i,j}^k = \frac{f(x_{i,j}^k) - f_{i,\min}}{f_{i,\max} - f_{i,\min}} \quad (6)$$

where $f_{i,\max}$ and $f_{i,\min}$ denote the maximum and minimum function values within niche N_i^k , respectively, and $f(x_{i,j}^k)$ is the function value of the individual. The normalized fitness $p_{i,j}^k$ ranges between 0 and 1. Using a threshold $p_f \in (0, 1)$, individuals with $p_{i,j}^k < p_f$ are considered fitter and are guided to converge toward nearby local optima, whereas those with $p_{i,j}^k \geq p_f$ are encouraged to explore new regions. The specific update operators applied to individuals based on their fitness levels are detailed below.

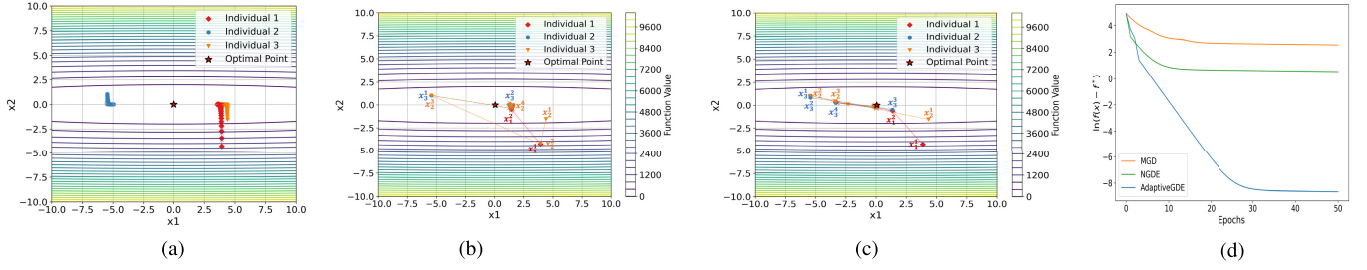


Fig. 2. Search trajectories and convergence behavior of MGD, NGDE, and AdaptiveGDE on the Bent Cigar function. Individual trajectories: (a) MGD, (b) NGDE, and (c) AdaptiveGDE. (d) Corresponding logarithmic error curves $\ln(f(x_{i,best}^k) - f^*)$.

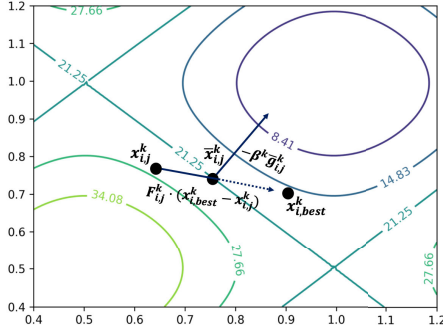


Fig. 3. Search directions of AdaptiveGDE overlaid on the contour lines of the Rastrigin function.

1) For an individual $x_{i,j}^k$ with higher fitness, the search is guided toward the best-performing member $x_{i,best}^k$ within niche N_i^k , using the direction $x_{i,best}^k - x_{i,j}^k$ to promote niche-level exploitation while maintaining global search diversity. The individual is first updated via a differential mutation step

$$\bar{x}_{i,j}^k = x_{i,j}^k + F_{i,j}^k \cdot (x_{i,best}^k - x_{i,j}^k) \quad (7)$$

which generates an intermediate point $\bar{x}_{i,j}^k$. The gradient $\bar{g}_{i,j}^k$ is then computed at this point, and the individual is further refined using gradient descent

$$x_{i,j}^{k+1} = \bar{x}_{i,j}^k - \eta_k \cdot \bar{g}_{i,j}^k. \quad (8)$$

Here, the negative-gradient direction offers the steepest descent, thereby improving local search efficiency. By integrating directional mutation with gradient-based refinement, AdaptiveGDE accelerates convergence within each niche.

Fig. 3 illustrates the search direction of the AdaptiveGDE algorithm on the contour plot of the 2-D Rastrigin function within the feasible region $[0.4, 1.2] \times [0.4, 1.2]$. The search direction is initially guided by the differential mutation vector $x_{i,best}^k - x_{i,j}^k$, producing the intermediate point $\bar{x}_{i,j}^k$. This point is then refined through a local update along the negative gradient direction $-\bar{g}_{i,j}^k$, which steers the search toward the valley (minimum) of the Rastrigin function.

In particular, let $x_{i,best}^k$ be the individual with the highest fitness within niche i at generation k . The offspring of this best individual is computed as

$$x_{i,best}^{k+1} = x_{i,best}^k - \eta_k \cdot g_{i,best}^k \quad (9)$$

where $\bar{x}_{i,best}^{k+1}$ represents the candidate offspring in generation $k + 1$, $-g_{i,best}^k$ is a descent direction, and η_k is the step size. If $-g_{i,best}^k$ is a good approximation of the negative gradient $-\nabla f(x_{i,best}^k)$, and η_k is appropriately small, then the objective function value is expected to decrease: $f(\bar{x}_{i,best}^{k+1}) < f(x_{i,best}^k)$. This mechanism underpins the convergence of the population by guiding the search toward regions of lower objective values.

2) Conversely, if an individual $x_{i,j}^k$ exhibits lower fitness, it is tasked with extending exploration within the feasible region to uncover better local optima. For this purpose, we apply the differential mutation operation (DE/rand/1) to mutate the individual as follows:

$$\bar{x}_{i,j}^k = x_{i,j}^k + F_{i,j}^k \cdot (x_1 - x_2) \quad (10)$$

$$x_{i,j}^{k+1} = \bar{x}_{i,j}^k - \eta_k \cdot \bar{g}_{i,j}^k \quad (11)$$

where $F_{i,j}^k > 0$ denotes the mutation factor, and x_1 and x_2 are two individuals randomly selected from the population P^k .

To determine the mutation factor $F_{i,j}^k > 0$, we adaptively adjust it based on individual fitness. Using upper and lower bounds (F_{max} and F_{min}), the factor is computed as

$$F_{i,j}^k = F_{min} + (F_{max} - F_{min}) \cdot p_{i,j}^k. \quad (12)$$

This adaptive strategy allows the algorithm to balance exploitation and exploration dynamically: individuals with better fitness receive smaller F -values to speed up convergence, while those with poorer fitness are assigned larger F -values to promote exploration.

D. Acceleration of the AdaptiveGDE Algorithm for Training Neural Networks

The objective function $f: \mathbb{R}^d \rightarrow \mathbb{R}$ for training a DNN is defined as the empirical loss over a set of training samples $\{\xi_1, \dots, \xi_n\}$ with loss function ℓ . To promote sparsity and reduce the computational burden, we introduce a differentiable sparsity regularizer [32], yielding

$$f(x) = \frac{1}{n} \sum_{j=1}^n \ell(x; \xi_j) + \lambda \sum_{i=1}^d \sqrt{x_i^2 + \alpha} \quad (13)$$

where $x \in \mathbb{R}^d$ denotes the model parameters, regularization parameter $\lambda > 0$, and $\alpha > 0$ is a smoothing constant.

Evaluating the full gradient over all n samples is computationally expensive, so we adopt mini-batch SGD. The training set is shuffled and partitioned into $\Gamma = \lceil n/m \rceil$ mini-batches of

size at most m . At iteration k , for a selected mini-batch ξ_b , the stochastic gradient at $x_{i,j}^k$ is computed as

$$g_{i,j}^{k,b-1} = \nabla f(x_{i,j}^k; \xi_b) = \frac{1}{m} \sum_{v=1}^m \nabla \ell(x_{i,j}^k; \xi_{b,v}) + \lambda \begin{bmatrix} \frac{x_1}{\sqrt{x_1^2 + \alpha}} \\ \vdots \\ \frac{x_d}{\sqrt{x_d^2 + \alpha}} \end{bmatrix}. \quad (14)$$

For large-scale DNN optimization, Adam often outperforms SGD due to its adaptive gradient updates [33]. Building on the estimated gradient defined in (14), we refine the gradient-accelerated differential mutation operator (see Section III-C), as detailed in the Supplementary Material Algorithm S.1 (see Appendix B). Individuals with better fitness follow the mutation strategy specified in line 7, while those with worse fitness adopt the rule in line 5. Subsequently, all individuals are updated using gradient information as described in lines 11–15.

The gradient is updated using exponential moving averages of the first and second moments

$$m_{i,j}^{k,b-1} = \beta_1 m_{i,j}^{k-1,b-1} + (1 - \beta_1) g_{i,j}^{k,b-1} \\ v_{i,j}^{k,b-1} = \beta_2 v_{i,j}^{k-1,b-1} + (1 - \beta_2) (g_{i,j}^{k,b-1})^2 \quad (15)$$

where $m_{i,j}^{k,b-1}$ and $v_{i,j}^{k,b-1}$ are the first- and second-order moment estimates of the gradient $g_{i,j}^{k,b-1}$, with decay rates β_1 and β_2 . Bias-corrected estimates are given by

$$\hat{m}_{i,j}^{k,b-1} = \frac{m_{i,j}^{k,b-1}}{1 - (1 - \beta_1)^k}, \quad \hat{v}_{i,j}^{k,b-1} = \frac{v_{i,j}^{k,b-1}}{1 - (1 - \beta_2)^k}. \quad (16)$$

The final update after differential mutation is

$$x_{i,j}^{k,b} \leftarrow x_{i,j}^{k,b-1} - \frac{\eta_k}{\sqrt{\hat{v}_{i,j}^{k,b-1} + \kappa}} \cdot \hat{m}_{i,j}^{k,b-1} \quad (17)$$

where η_k is the learning rate, and κ is a small constant to prevent division by zero.

In contrast to GEMONN [14], which uses gradient information implicitly to guide mutation, and NGDE, which directly adds the negative gradient to the differential mutation vector, AdaptiveGDE introduces a two-stage update scheme within the mutation process. Specifically, differential mutation is first applied to generate an intermediate solution that maintains population diversity. Gradient descent (or multiple SGD steps) is then performed from this intermediate point to refine the search direction and enhance local convergence. This decoupled strategy contrasts with GEOMMN's implicit guidance and NGDE's direct additive updates, addressing their respective drawbacks: slow convergence and unbalanced, unstable updates in complex landscapes. By applying gradient steps after mutation rather than during it, AdaptiveGDE achieves a more effective balance between exploration and exploitation when optimizing nonconvex functions or DNNs.

IV. CONVERGENCE ANALYSIS

Convergence analysis of gradient-accelerated EAs remains largely underexplored in the existing literature. These algorithms inherently require the introduction of sparse individuals

and simplified selection operations, which introduce additional challenges in the form of nonsmooth regularization and inherent randomness.

- 1) *Nonsmooth Regularization*: To reduce storage overhead, EAs often employ sparsity-promoting norms, such as the ℓ_1 norm, to encourage the generation of sparse individuals. However, these nonsmooth regularizers violate the standard smoothness assumptions, such as Lipschitz continuity, that are typically required for analyzing the convergence of gradient-based optimization methods.
- 2) *Randomness in the Selection Operation*: Unlike previous gradient-based hybrid approaches that rely on explicit selection strategies, AdaptiveGDE incorporates mutated individuals directly into the new population. Although this design reduces computational cost, it complicates theoretical analysis because the identity of the best-performing individual may change unpredictably across generations.

To address these challenges, we adopt two strategies. For 1), we relax the smoothness condition (see Assumption 2) and exploit the structure of the gradient-based mutation operator to handle nonsmooth regularization. For 2), we employ a two-stage mutation operator that tracks the best individual x_{best}^k in each generation, updating it solely via gradient-based steps (e.g., Adam). Under this framework and the relaxed smoothness assumption, we prove that x_{best}^k converges to an ϵ -approximate stationary point at an $O(1/\epsilon^2)$ rate with high probability, even under approximate ℓ_1 regularization. This establishes a theoretical foundation for analyzing gradient-accelerated EAs with sparsity-inducing regularization.

Suppose the training dataset is randomly partitioned into Γ mini-batches: $\xi_1, \xi_2, \dots, \xi_\Gamma$. The gradient of the objective function f at $x_{i,j}^{k,b}$ is estimated using one such mini-batch, as described in (14). The analysis is based on the following conditions.

Assumption 1: The objective function f is differentiable and closed over its open domain $\text{dom}(f) \subseteq \mathbb{R}^d$, and is bounded from below; that is, $f^* := \inf_x f(x) > -\infty$.

Assumption 2 [34]: Let f be a differentiable real-valued function. There exist constants $\rho, L_0, L_\rho \geq 0$ such that, almost everywhere in $\text{dom}(f)$, the following inequality holds:

$$\|\nabla^2 f(x)\| \leq L_0 + L_\rho \|\nabla f(x)\|^\rho$$

where $0 \leq \rho < 2$. That is, the function f is (ρ, L_0, L_ρ) -smooth.

Remark 1: Assumption 2 imposes a relaxed smoothness condition, where the Hessian norm is bounded by a linear combination of the gradient raised to the power $\rho \in [0, 2)$. Unlike local strong convexity, which requires a positive definite Hessian, this assumption allows curvature to vary with the gradient's magnitude, enabling (ρ, L_0, L_ρ) -smoothness.

Remark 2: The empirical loss term in (13), $1/q \sum_{j=1}^q \ell(x; \xi_j)$, is typically twice differentiable (e.g., cross-entropy). The regularization term, $\lambda \sum_{i=1}^d (x_i^2 + \alpha)^{1/2}$, is a smooth approximation of the ℓ_1 -norm with uniformly bounded second derivatives when $\alpha > 0$. Thus, $f(x)$ satisfies the (ρ, L_0, L_ρ) -smoothness condition with $\rho \in [0, 2)$, as it is twice differentiable almost everywhere and its Hessian norm grows at most polynomially with the gradient norm.

Assumption 3: The gradient noise is centered and almost surely bounded. In particular, for some $\sigma \geq 0$ and all $k \geq 1$

$$\begin{aligned}\mathbb{E}_{\Gamma-1} \left[g_{best}^{k+1, \Gamma} \right] &= \nabla f(x_{best}^k) \\ \left\| g_{best}^{k+1, \Gamma} - \nabla f(x_{best}^k) \right\| &\leq \sigma\end{aligned}$$

where $\mathbb{E}_{\Gamma-1}[\cdot] := \mathbb{E}[\cdot \mid \xi_1, \dots, \xi_{\Gamma-1}]$ is the conditional expectation given $\xi_1, \dots, \xi_{\Gamma-1}$.

Assumption 3 is a standard condition in the convergence analysis of gradient-based algorithms (e.g., [33], [34], [35]). By combining this bounded-noise assumption with the relaxed smoothness condition of Assumption 2, we establish the convergence properties of the best-evolved individual in AdaptiveGDE, as illustrated in Lemma 1 and Theorem 1. For notational simplicity, we write $x_{best}^{k, \Gamma}$ and $g_{best}^{k, \Gamma}$ as x_{best}^k and g_{best}^k , respectively.

Lemma 1: Suppose Assumptions 1–3 hold. Choose G to be a constant satisfying $G \geq \sigma$. Define

$$\begin{aligned}r &:= \min \left\{ \frac{1}{5L_p G^{\rho-1}}, \frac{1}{5 \left(L_0^{\rho-1} L_p \right)^{1/\rho}} \right\} \\ L &:= 3L_0 + 4L_p G^\rho, \quad D = 2G/\kappa.\end{aligned}$$

Suppose that the AdaptiveGDE algorithm runs with a fixed step size, $\eta_k = \bar{\eta}$ for all $k \in \mathbb{N}$, satisfying $\bar{\eta} \leq \min\{r/D, \kappa/6L\}$. Denote

$$\epsilon_{best}^k := \hat{m}_{best}^k - \nabla f(x_{best}^k)$$

as the deviation of the re-scaled momentum from the actual gradient $\nabla f(x_{best}^k)$. The change in the evolved function values of the best individual is as follows:

$$f(x_{best}^{k+1}) - f(x_{best}^k) \leq -\frac{\bar{\eta}}{4G} \left\| \nabla f(x_{best}^k) \right\|^2 + \frac{\bar{\eta}}{\kappa} \left\| \epsilon_{best}^k \right\|^2.$$

If $\left\| \epsilon_{best}^k \right\|^2 \leq \kappa/4G \left\| \nabla f(x_{best}^k) \right\|^2$, then $f(x_{best}^{k+1})$ decreases compared to $f(x_{best}^k)$.

Remark 3: Lemma 1 bounds the improvement in function value for the optimal individuals generated by AdaptiveGDE, based on the following.

- 1) The squared norm of the full gradient at the current optimal point $\nabla f(x_{best}^k)$, where a larger gradient leads to greater value reduction.
- 2) The deviation of the corrected momentum estimate $\hat{m}_{best}^k$ from the actual gradient $\nabla f(x_{best}^k)$, reflecting the impact of inaccurate momentum estimation. This estimate serves as the basis for the convergence analysis of AdaptiveGDE.

Theorem 1: Suppose Assumptions 1–3 hold. Define c_1, c_2 as small positive scalars, C_1, C_2 as large positive scalars, and let $\varepsilon > 0$ be a predetermined small scalar. Let the decay rates for the first- and second-moment gradient estimates satisfy

$$\beta_1 \leq \min \left\{ 1, \frac{c_1 \kappa \varepsilon^2}{\sigma^2 G \sqrt{\iota}} \right\}, \quad 0 \leq \beta_2 \leq 1$$

where $\iota := \log(1/\delta)$ for any $0 < \delta < 1$. Define the initial suboptimality gap as $\Delta^1 = f(x_{best}^1) - f^* < \infty$, and let G be a constant such that

$$G \geq \max \left\{ 2\kappa, 2\sigma, \sqrt{C_1 \Delta^1 L_0}, (C_1 \Delta^1 L_p)^{\frac{1}{2-\rho}} \right\}.$$

Furthermore, define $r := \min\{1/(5L_p G^{\rho-1}), 1/(5(L_0^{\rho-1} L_p)^{1/\rho})\}$, $L := 3L_0 + 4L_p G^\rho$. Assume the AdaptiveGDE runs with a fixed step size $\eta_k = \bar{\eta}$ for $k \geq 1$, where $\bar{\eta}$ satisfies

$$\bar{\eta} \leq c_2 \cdot \min \left\{ \frac{r\kappa}{G}, \frac{\sigma\kappa\beta_1}{LG\sqrt{\iota}}, \frac{\kappa^{3/2}\beta_1}{L\sqrt{G}} \right\}.$$

Let

$$K = \max \left\{ \frac{1}{\beta_1^2}, \frac{C_2 \Delta^1 G}{\bar{\eta} \varepsilon^2} \right\}.$$

Then, with probability at least $1 - \delta$, we have

$$\left\| \nabla f(x_{best}^k) \right\| \leq G \quad \text{for all } 1 \leq k \leq K$$

and

$$\frac{1}{K} \sum_{k=1}^K \left\| \nabla f(x_{best}^k) \right\|^2 \leq \varepsilon^2.$$

Remark 4: Theorem 1 yields the following insights.

- 1) The bound $1/K \sum_{k=1}^K \left\| \nabla f(x_{best}^k) \right\|^2 \leq \varepsilon^2$ guarantees convergence to an ε -stationary point in terms of the average squared gradient norm.
- 2) With $\beta_1 = \mathcal{O}(\varepsilon^2)$ and $\bar{\eta} \propto \beta_1$, the resulting iteration complexity satisfies $K = \mathcal{O}(\varepsilon^{-4})$.

Appendices C-A–C-C in the supplementary material provide all proofs and detailed technical derivations. In Appendix C-C (see supplementary material), we further discuss the convergence analysis of AdaptiveGDE, offering detailed explanations of Assumption 3, Lemma 1, and Theorem 1. Building on the theoretical bounds established in Theorem 1, Appendix C-E (see supplementary material) presents empirical sensitivity studies to investigate the practical impact of architectural depth (size) and the numerical stabilizer κ in (17).

V. NUMERICAL EXPERIMENTS

To assess AdaptiveGDE's performance, we tested it on multimodal function minimization and DNN training tasks. The experiments were executed in Python 3.9 on a server with 16 vCPUs (Intel¹ Xeon¹ Platinum 8481C) and an NVIDIA GeForce RTX 4090 GPU.

A. Minimization of Multimodal Functions

We evaluated AdaptiveGDE on nine multimodal benchmark functions with dimensions ranging from 10 to 50 000. Their properties are summarized in Table S.4, and landscape visualizations are provided in Fig. S.1 (see the Supplementary Material).

We compare AdaptiveGDE with two categories of baselines.

- 1) *EAs:* Including the genetic algorithm (GA), several DE variants (DE, JADE, SADE, SAP_DE, SHADE, and L_SHADE [36]), and ESs: the original ES, Levy ES, CMA-ES [36], MAP-CMA-ES [37], and LRA-CMA-ES [38]. Due to the high computational and memory cost of covariance-matrix adaptation, CMA-ES, MAP-CMA-ES, and LRA-CMA-ES were only tested on problems with dimensionality below 1000.

¹Registered trademark.

TABLE I

WILCOXON RANK-SUM TEST RESULTS AT THE 0.05 SIGNIFICANCE LEVEL. THE SYMBOLS “+,” “−,” AND “≈” DENOTE THAT ADAPTIVEGDE PERFORMS SIGNIFICANTLY BETTER, SIGNIFICANTLY WORSE, OR SHOWS NO SIGNIFICANT DIFFERENCE COMPARED TO THE BASELINE, RESPECTIVELY

Function	GA	DE	JADE	SADE	SAP_DE	SHADE	L_SHADE	OriginalES	LevyES	CMA_ES	MAP_CMA_ES	LRA_CMA_ES	MPGD	MGD	ESGD	EGD	NGDE
Ackley	8/0/0	7/0/1	7/0/1	8/0/0	8/0/0	7/0/1	7/0/1	8/0/0	8/0/0	4/0/0	3/0/1	4/0/0	8/0/0	8/0/0	8/0/0	8/0/0	0/7/1
Bent Cigar	8/0/0	8/0/0	8/0/0	8/0/0	8/0/0	8/0/0	8/0/0	8/0/0	8/0/0	4/0/0	4/0/0	4/0/0	8/0/0	8/0/0	8/0/0	8/0/0	8/0/0
Griewank	8/0/0	8/0/0	8/0/0	8/0/0	8/0/0	8/0/0	8/0/0	8/0/0	8/0/0	4/0/0	4/0/0	4/0/0	8/0/0	8/0/0	8/0/0	8/0/0	8/0/0
Levy	7/0/1	7/0/1	7/0/1	7/0/1	8/0/0	7/0/1	7/0/1	7/0/1	8/0/0	3/0/1	4/0/0	4/0/0	8/0/0	8/0/0	8/0/0	8/0/0	2/0/6
Rastrigin	8/0/0	8/0/0	8/0/0	8/0/0	7/0/1	7/0/1	8/0/0	8/0/0	8/0/0	4/0/0	4/0/0	4/0/0	8/0/0	8/0/0	8/0/0	8/0/0	8/0/0
Rosenbrock	8/0/0	8/0/0	8/0/0	8/0/0	1/0/7	8/0/0	8/0/0	8/0/0	8/0/0	4/0/0	4/0/0	4/0/0	7/0/1	8/0/0	7/1/0	6/0/2	7/0/1
Shubert	7/0/1	8/0/0	8/0/0	8/0/0	4/0/4	7/0/1	8/0/0	8/0/0	8/0/0	4/0/0	4/0/0	4/0/0	7/1/0	7/1/0	6/0/2	8/0/0	8/0/0
Sphere	8/0/0	8/0/0	8/0/0	8/0/0	8/0/0	8/0/0	8/0/0	8/0/0	8/0/0	4/0/0	4/0/0	4/0/0	8/0/0	8/0/0	8/0/0	8/0/0	0/0/8
Styblinski-Tang	8/0/0	7/0/1	8/0/0	7/0/1	4/0/4	8/0/0	8/0/0	8/0/0	8/0/0	4/0/0	4/0/0	4/0/0	2/0/6	2/0/6	2/6/0	3/0/5	3/0/5
+ / ≈ / −	70/0/2	69/0/3	70/0/2	70/0/2	56/0/16	68/0/4	70/0/2	71/0/1	72/0/0	35/0/1	35/0/1	36/0/0	64/1/7	65/1/6	63/7/2	65/0/7	44/7/21

2) *Gradient-Based Hybrid Algorithms*: Including MGD and MPGD [30], as well as ESGD [12], EGD [21], and NGDE [15]. ESGD, EGD, and NGDE combine evolutionary mechanisms with gradient-based updates, whereas MGD and MPGD run multiple independent instances of GD or PGD and return the best solution. Detailed parameter settings are provided in Appendix D-A (see the Supplementary Material).

Comprehensive results for all nine test functions, evaluated over dimensions ranging from 10 to 50 000, are reported in the Supplementary Material Tables S.5–S.20 and Figs. S.2–S.17 in Appendix D-B. To assess statistical significance, we conducted the Wilcoxon rank-sum test [39] at the 0.05 level. Symbols “+,” “−,” and “≈” denote that AdaptiveGDE performed significantly better, worse, or similar to each baseline, respectively. The detailed results are presented in Table I, covering all nine functions across all evaluated dimensions.

We observe the following from the results.

- 1) *Superior Global Exploration*: On multimodal benchmark functions with many local minima (e.g., Ackley, Rastrigin, Shubert), AdaptiveGDE consistently outperforms both evolutionary and gradient-based hybrid methods, achieving the lowest best objective values and the most “+” outcomes in the Wilcoxon test. Notably, on challenging Rastrigin and high-dimensional Shubert functions, known for sharp oscillations, AdaptiveGDE significantly outperforms all competitors, demonstrating strong global exploration.
- 2) *Strong Local Exploitation*: Convex functions, such as Bent Cigar and Sphere, test local convergence. On Bent Cigar, AdaptiveGDE attains the best and mean objective values below $1.0\text{E-}11$ across most dimensions, nearly reaching the global optimum (0) and outperforming all others. On Sphere, it matches NGDE in performance and surpasses other methods in the best objective value, highlighting superior local exploitation. Overall, AdaptiveGDE consistently delivers robust, high-quality solutions across all tested multimodal functions, maintaining performance even as dimensionality increases.

1) *Runtime Evaluation*: Figs. S.18–S.21 in the supplementary material report elapsed times on multimodal functions with dimensions 10, 100, 1000, and 10 000. AdaptiveGDE

is competitive with, and often faster than, baseline methods at lower dimensions; at 10 000 dimensions its runtime is comparable to that of gradient-based hybrids and slightly higher than that of traditional EAs, owing to the additional cost of gradient computations.

We also report function evaluations (FEs) as a standard efficiency metric. Because AdaptiveGDE additionally incurs gradient-computation costs, Table S.21 in the supplementary material details both gradient-descent steps and fitness evaluations for each algorithm. A detailed analysis is provided in Appendix D-C of the supplementary material.

B. Training DNNs

We evaluated AdaptiveGDE for training DNNs on several benchmark datasets, comparing it against two categories of optimizers.

- 1) *Gradient-Based Optimizers*: M_SGD, M_RMSprop, M_Adam [1], M_AdamW [20], M_RAdam [19], M_NAdam [4], M_Lookahead_SGD, M_Lookahead_Adam, M_Lookahead_AdamW [6], M_Lion [7], and M_Adan [8]. Each method runs the corresponding optimizer independently μ times with different random initializations, and the best-performing model across runs is selected.
- 2) *Gradient-Based Hybrid EAs*: ESGD_SGD, ESGD_Adam [12], NGDE_SGD, and NGDE_Adam [15]. Specifically, ESGD_Adam, NGDE_Adam, and AdaptiveGDE use Adam for gradient updates, while ESGD_SGD and NGDE_SGD employ standard SGD. For the evolutionary component of ESGD, DE is used to ensure a fair comparison.

Full experimental settings are provided in Appendix E-A of the supplementary material. Detailed descriptions of the datasets and DNN architectures are given in Tables S.22 and S.23, and the sizes of training, validation, and test sets are listed in Table S.24 (see the supplementary material).

Table II reports the validation loss, test loss, and classification accuracy for ResNet-18 trained with various optimizers on CIFAR-10 and SVHN under various training set sizes. Tables S.25–S.36 in the supplementary material appendix summarize the achieved validation loss, test loss, and accuracy

TABLE II
COMPARISON OF TESTED ALGORITHMS ON RESNET-18 ACROSS CIFAR-10 AND SVHN UNDER VARYING TRAINING SIZES

Datasets	Algorithm type	Algorithm	Training size = 5000			Training size = 10000			Training size = 20000		
			Valid Cost	Test Acc (%)	Test Cost	Valid Cost	Test Acc (%)	Test Cost	Valid Cost	Test Acc (%)	Test Cost
CIFAR-10	Mutil-GDs	M_SGD	2.4693	37.88	2.4876	2.4068	41.39	2.5286	2.7241	46.62	2.5708
		M_RMSprop	2.0340	56.16	2.2281	2.0359	62.27	2.0721	1.8306	67.66	1.7417
		M_Adam	2.6284	55.14	2.6547	2.0052	61.85	2.1123	1.6850	68.73	1.6779
		M_AdamW	2.8635	56.70	2.8122	1.9973	63.37	2.3017	2.1490	70.78	2.1775
		M_RADAM	2.3067	56.86	2.3407	1.8302	64.58	1.8464	1.4948	70.22	1.5356
		M_NADAM	2.4124	56.00	2.3711	1.8137	63.71	1.8662	1.6283	70.13	1.6278
		M_Lookahead_SGD	2.7605	43.70	2.9418	2.6988	47.42	2.8115	1.9210	67.29	1.8968
		M_Lookahead_ADAM	2.7433	55.60	2.5168	2.1166	60.96	2.2141	2.6872	53.24	2.6694
		M_Lookahead_ADAMW	2.8581	56.52	2.7895	2.3635	63.41	2.4359	1.8263	68.11	1.7423
	Hybrids	M_Lion	3.5801	54.16	3.5625	3.1418	60.77	3.1044	2.3007	69.68	2.2013
		M_Adan	3.8037	54.24	3.5932	2.8793	60.79	3.2266	2.7605	67.22	2.5151
		ESGD_SGD	4.3073	37.70	4.4252	3.1482	48.69	3.2226	3.0803	54.69	2.9325
		ESGD_ADAM	3.2983	53.94	3.2735	2.3218	59.63	2.4542	1.9210	67.29	1.8968
		NGDE_SGD	3.0540	48.10	2.8213	2.2378	51.05	2.4747	2.5392	49.11	2.5147
		NGDE_ADAM	2.2557	54.28	2.1471	1.9069	64.51	1.9724	1.7365	68.85	1.6512
		AdaptiveGDE	1.3706	60.74	1.3514	1.1406	64.15	1.2819	1.0011	69.46	0.9545
SVHN	Mutil-GDs	M_SGD	1.7349	52.02	1.8656	1.0504	77.02	1.1796	0.8744	82.75	0.9290
		M_RMSprop	0.6528	84.12	0.7234	0.9299	81.10	1.1080	0.5147	88.54	0.5674
		M_Adam	0.7229	84.28	0.7873	0.6884	83.10	0.7832	0.5236	88.59	0.5464
		M_AdamW	0.8333	84.08	0.8962	1.0848	84.19	1.2638	0.6950	87.51	0.6703
		M_RADAM	0.5821	84.88	0.7427	0.7297	83.96	0.7933	0.5760	87.73	0.6220
		M_NADAM	0.6708	83.34	0.8101	0.5772	86.35	0.6786	0.5800	88.15	0.6427
		M_Lookahead_SGD	1.2880	71.30	1.3293	0.9724	76.43	1.1342	0.8748	81.70	0.9294
		M_Lookahead_ADAM	0.7985	82.94	0.8322	0.5645	86.77	0.6586	0.5125	88.92	0.5410
		M_Lookahead_ADAMW	0.7933	82.96	1.0163	0.7889	87.35	0.8376	0.8234	89.90	0.8857
	Hybrids	M_Lion	1.0799	82.04	1.3818	0.9874	85.05	1.2010	0.7658	88.46	0.8228
		M_Adan	1.2105	80.88	1.2726	1.0076	84.35	1.2113	0.8738	88.40	0.8422
		ESGD_SGD	1.9557	68.84	2.0494	1.5503	75.89	1.7387	1.3678	82.54	1.4662
		ESGD_ADAM	1.0551	83.22	1.1937	0.9003	84.72	0.9515	0.7824	87.98	0.7553
		NGDE_SGD	1.5218	56.88	1.6105	1.3074	68.58	1.2824	0.7842	77.60	0.8427
		NGDE_ADAM	0.7206	80.96	0.7444	0.5558	86.12	0.6892	0.3910	88.42	0.4352
		AdaptiveGDE	0.4436	84.74	0.5184	0.4289	86.27	0.5054	0.3897	88.55	0.4071

for LeNet-5, ResNet-18, ResNet-50, and DenseNet-121 on MNIST, FashionMNIST, CIFAR-10, CIFAR-100, and SVHN with varying training set sizes. We also evaluated Vision Transformer models, with detailed results for ViT-tiny and ViT-small on CIFAR-10 provided in Appendix E-C (see the supplementary material).

1) *Statistical Analysis With the Nemenyi Test:* We evaluated each algorithm by ranking its performance on every dataset, assigning higher ranks to lower losses and higher accuracies. These ranks were then averaged to assess overall effectiveness. Statistical significance was evaluated using p -values and validated through the Nemenyi test.

Fig. 4 shows the average ranks under the minimal training size, along with the Nemenyi test's critical difference at $p = 0.05$. The results indicate that AdaptiveGDE significantly outperforms competitors in both test accuracy and loss. On the minimal training sets, it achieves the top average rank in test loss (1.0), surpassing the second-best, M_RMSprop, by 4.923 ranks, and leads in test accuracy (1.577), exceeding the second-best, M_RADAM, by 1.538 ranks. Overall, AdaptiveGDE demonstrates clear advantages under limited training data. Additional results for a training size of 10000, as well

as for all training sizes, are provided in the supplementary material Figs. S.22 and S.23.

2) *Role of the Adaptive Gradient-Accelerated Differential Mutation Operator:* The adaptive gradient-accelerated differential mutation operator updates individuals according to their fitness. High-fitness individuals follow the direction of the niche's best solution for local exploration. Low-fitness individuals undergo randomized, differential mutations to promote diversity and broad exploration. After mutations produce intermediate candidates, all individuals incorporate gradient information to accelerate convergence. To isolate the contributions of each mutation operator, two ablation variants are introduced: "AdaptiveGDEV1," which omits directed mutation of the niche's best solution for high-fitness individuals, and "AdaptiveGDEV2," which removes randomized exploration for low-fitness individuals.

Table III and Table S.39 (see Appendix E-D1 in the supplementary material) demonstrate that AdaptiveGDE consistently outperforms both variants. With "AdaptiveGDEV1," which lacks the directed mutation of the best individual, LeNet-5's validation loss increases by 9.81%, test loss by 77.7%, and accuracy drops by 0.44%. On ResNet-18 with color images, validation loss rises by up to 93%, and accuracy falls by an

baseline optimizers and ablation studies on the smoothing parameter α in (13).

5) *Runtime Evaluation*: Fig.S.34 in the supplementary material shows ResNet-18 training seconds across various datasets. Overall, the five evolutionary-gradient hybrid algorithms have slightly higher training times than traditional gradient-based methods. Among them, AdaptiveGDE trains marginally faster than ESGD and is comparable to NGDE.

VI. CONCLUSION

This study presents the AdaptiveGDE algorithm to tackle high-dimensional nonconvex optimization problems. To effectively balance exploration and exploitation in such complex landscapes, AdaptiveGDE introduces a gradient-accelerated differential mutation operator, which decouples the gradient descent component from DE. In addition, an adaptive niching strategy is employed to dynamically adjust niche sizes throughout the optimization process. Theoretical results provide a solid foundation for designing a gradient-accelerated EA framework that incorporates sparsity-regularized individuals. The analysis demonstrates that, under appropriate parameter settings, relaxed smoothness conditions, and assuming centered and almost surely bounded gradient noise, AdaptiveGDE guarantees that, with high probability, the gradient norm of the best iterate remains bounded at each step. Furthermore, the average squared gradient norm over K iterations remains bounded by a specified threshold ε^2 . The theoretical analysis provides a general framework for studying EAs that integrate gradient-accelerated mutation operators with various sparsity-inducing regularization terms.

Extensive experiments demonstrate the effectiveness of AdaptiveGDE as follows.

- 1) It exhibits strong global exploration, particularly on highly multimodal functions with sharp variations and numerous local minima.
- 2) On convex quadratic functions, such as Bent Cigar and Sphere, it shows strong local exploitation capabilities.
- 3) When applied to DNN training, AdaptiveGDE significantly improves test accuracy and reduces test loss, especially with limited training data.

Despite its strengths, AdaptiveGDE does not consistently improve test accuracy on high-dimensional DNN models, highlighting a trade-off between cost reduction and classification performance. Its population-based design also incurs higher computational and memory costs. Future work will focus on refining gradient-guided evolutionary updates, developing joint objectives that balance cost and accuracy, and exploring model sparsification and first-order gradient-acceleration techniques to reduce memory usage and improve efficiency while maintaining performance.

REFERENCES

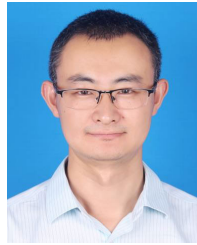
- [1] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2015, doi: [10.48550/arXiv.1412.6980](https://doi.org/10.48550/arXiv.1412.6980).
- [2] A. Botev, G. Lever, and D. Barber, "Nesterov's accelerated gradient and momentum as approximations to regularised update descent," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, May 2017, pp. 1899–1903.
- [3] J. Zhuang et al., "AdaBelief optimizer: Adapting stepsizes by the belief in observed gradients," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 18795–18806, doi: [10.5555/3495724.3497302](https://doi.org/10.5555/3495724.3497302).
- [4] T. Dozat, "Incorporating Nesterov momentum into Adam," in *Proc. ICLR Workshop*, 2016. [Online]. Available: <https://openreview.net/pdf?id=OM0jvwB8jIp57ZJtNEZ>
- [5] P. Zhou, X. Xie, and S. Yan, "Win: Weight-decay-integrated Nesterov acceleration for adaptive gradient algorithms," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2023.
- [6] M. Zhang, J. Lucas, J. Ba, and G. E. Hinton, "Lookahead optimizer: K steps forward, 1 step back," in *Proc. Adv. Neural Inf. Process. Syst.*, pp. 9593–9604, 2019.
- [7] X. Chen et al., "Symbolic discovery of optimization algorithms," in *Proc. Adv. Neural Inf. Process. Syst.*, 2023, pp. 49205–49233.
- [8] X. Xie, P. Zhou, H. Li, Z. Lin, and S. Yan, "Adan: Adaptive Nesterov momentum algorithm for faster optimizing deep models," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 46, no. 12, pp. 9508–9520, Dec. 2024.
- [9] M. Biaoletti, G. Di Bari, A. Milani, and V. Poggioni, "Differential evolution for neural networks optimization," *Mathematics*, vol. 8, no. 1, p. 69, Jan. 2020.
- [10] M. Agarwal, S. K. Gupta, and K. K. Biswas, "DECACNN: Differential evolution-based approach to compress and accelerate the convolution neural network model," *Neural Comput. Appl.*, vol. 36, no. 6, pp. 2665–2681, Feb. 2024.
- [11] H. Zhao, C. Zhang, B. Xue, M. Zhang, and B. Zhang, "A two-stage differential evolutionary algorithm for deep ensemble model generation," *IEEE Trans. Evol. Comput.*, vol. 28, no. 3, pp. 810–824, Jun. 2024.
- [12] X. Cui, Z. We, Z. Tüske, and M. Picheny, "Evolutionary stochastic gradient descent for optimization of deep neural networks," in *Proc. Adv. Neural Inf. Process. Syst. (NIPS)*, vol. 31, 2018, pp. 6048–6058.
- [13] Y. Xue, Y. Tong, and F. Neri, "A hybrid training algorithm based on gradient descent and evolutionary computation," *Int. J. Speech Technol.*, vol. 53, no. 18, pp. 21465–21482, Sep. 2023.
- [14] S. Yang, Y. Tian, C. He, X. Zhang, K. C. Tan, and Y. Jin, "A gradient-guided evolutionary approach to training deep neural networks," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 33, no. 9, pp. 4861–4875, Sep. 2022.
- [15] Q. Yu, X. Liang, M. Li, and L. Jian, "NGDE: A niching-based gradient-directed evolution algorithm for nonconvex optimization," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 36, no. 3, pp. 5363–5374, Mar. 2025.
- [16] N. Qian, "On the momentum term in gradient descent learning algorithms," *Neural Netw.*, vol. 12, no. 1, pp. 145–151, Jan. 1999.
- [17] L. Luo, Y. Xiong, Y. Liu, and X. Sun, "Adaptive gradient methods with dynamic bound of learning rate," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2019, doi: [10.48550/arXiv.1902.09843](https://doi.org/10.48550/arXiv.1902.09843).
- [18] B. Heo et al., "AdamP: Slowing down the slowdown for momentum optimizers on scale-invariant weights," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2021, doi: [10.48550/arXiv.2006.08217](https://doi.org/10.48550/arXiv.2006.08217).
- [19] L. Liu et al., "On the variance of the adaptive learning rate and beyond," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2020, doi: [10.48550/arXiv.1908.03265](https://doi.org/10.48550/arXiv.1908.03265).
- [20] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2019, doi: [10.48550/arXiv.1711.05101](https://doi.org/10.48550/arXiv.1711.05101).
- [21] K. Xue, C. Qian, L. Xu, and X. Fei, "Evolutionary gradient descent for non-convex optimization," in *Proc. 30th Int. Joint Conf. Artif. Intell.*, Aug. 2021, pp. 3221–3227.
- [22] E. Real, A. Aggarwal, Y. Huang, and Q. V. Le, "Regularized evolution for image classifier architecture search," in *Proc. AAAI Conf. Artif. Intell.*, 2019, pp. 4780–4789.
- [23] Z. Wei, W. Gao, G. Li, and Q. Zhang, "A penalty-based differential evolution for multimodal optimization," *IEEE Trans. Cybern.*, vol. 52, no. 7, pp. 6024–6033, Jul. 2022.
- [24] B. Y. Qu, P. N. Suganthan, and J. J. Liang, "Differential evolution with neighborhood mutation for multimodal optimization," *IEEE Trans. Evol. Comput.*, vol. 16, no. 5, pp. 601–614, Oct. 2012.
- [25] Q. Yang, W.-N. Chen, Y. Li, C. L. P. Chen, X.-M. Xu, and J. Zhang, "Multimodal estimation of distribution algorithms," *IEEE Trans. Cybern.*, vol. 47, no. 3, pp. 636–650, Mar. 2017.
- [26] Z.-J. Wang et al., "Automatic niching differential evolution with contour prediction approach for multimodal optimization problems," *IEEE Trans. Evol. Comput.*, vol. 24, no. 1, pp. 114–128, Feb. 2020.
- [27] J. Liang, G. Chen, B. Qu, C. Yue, K. Yu, and K. Qiao, "Niche-based cooperative co-evolutionary ensemble neural network for classification," *Appl. Soft Comput.*, vol. 113, Dec. 2021, Art. no. 107951.
- [28] W. Sheng, X. Wang, Z. Wang, Q. Li, Y. Zheng, and S. Chen, "A differential evolution algorithm with adaptive niching and K-means operation for data clustering," *IEEE Trans. Cybern.*, vol. 52, no. 7, pp. 6181–6195, Jul. 2022.

- [29] Q. Yang et al., "Triple competitive differential evolution for global numerical optimization," *Swarm Evol. Comput.*, vol. 84, Feb. 2024, Art. no. 101450.
- [30] C. Jin, R. Ge, P. Netrapalli, S. M. Kakade, and M. I. Jordan, "How to escape saddle points efficiently," in *Proc. 34th Int. Conf. Mach. Learn.*, 2017, pp. 1724–1732.
- [31] Y. Xue, Y. Tong, and F. Neri, "An ensemble of differential evolution and Adam for training feed-forward neural networks," *Inf. Sci.*, vol. 608, pp. 453–471, Aug. 2022.
- [32] Y. Wu, Y. Lan, L. Zhang, and Y. Xiang, "Feature flow regularization: Improving structured sparsity in deep neural networks," *Neural Netw.*, vol. 161, pp. 598–613, Apr. 2023.
- [33] J. Zhang, T. He, S. Sra, and A. Jadbabaie, "Why gradient clipping accelerates training: A theoretical justification for adaptivity," in *Proc. Int. Conf. Learn. Represent (ICLR)*, 2020, doi: [10.48550/arXiv.1905.11881](https://doi.org/10.48550/arXiv.1905.11881).
- [34] A. Jadbabaie, H. Li, and A. Rakhlin, "Convergence of Adam under relaxed assumptions," in *Proc. Adv. Neural Inf. Process. Syst.*, 2023, pp. 52166–52196.
- [35] A. Jadbabaie, H. Li, J. Qian, A. Rakhlin, and Y. Tian, "Convex and non-convex optimization under generalized smoothness," in *Proc. Adv. Neural Inf. Process. Syst.*, 36, 2023, pp. 40238–40271.
- [36] A. W. Mohamed, A. A. Hadi, and K. M. Jambi, "Novel mutation strategy for enhancing SHADE and LSHADE algorithms for global numerical optimization," *Swarm Evol. Comput.*, vol. 50, Nov. 2019, Art. no. 100455.
- [37] R. Hamano, S. Shirakawa, and M. Nomura, "Natural gradient interpretation of rank-one update in CMA-ES," in *Proc. Int. Conf. Parallel Problem Solving Nature (PPSN)*. Cham, Switzerland: Springer, 2024, pp. 252–267.
- [38] M. Nomura, Y. Akimoto, and I. Ono, "CMA-ES with learning rate adaptation," *ACM Trans. Evol. Learn. Optim.*, vol. 5, no. 1, pp. 1–28, Mar. 2025.
- [39] J. Derrac, S. García, D. Molina, and F. Herrera, "A practical tutorial on the use of nonparametric statistical tests as a methodology for comparing evolutionary and swarm intelligence algorithms," *Swarm Evol. Comput.*, vol. 1, no. 1, pp. 3–18, Mar. 2011.



Qi Yu received the B.Sc. degree in engineering management from Shandong Normal University, Jinan, China, in 2020, and the M.Sc. degree in applied statistics from China University of Petroleum, Qingdao, China, in 2023, where she is currently pursuing the Ph.D. degree in applied mathematics and energy data science.

Her scientific interests include deep learning and nonconvex optimization.



Xijun Liang (Member, IEEE) received the B.Sc. degree in mathematics from China University of Petroleum, Dongying, China, in 2007, and the Ph.D. degree in operational research and cybernetics from Dalian University of Technology, Dalian, China, in 2013.

Since 2014, he has been with China University of Petroleum, Qingdao, China, where he is currently an Associate Professor with the College of Science. His current research interests include large-scale optimization and machine learning.



Jinmeng Liu received the B.Sc. degree in engineering management from Shandong Normal University, Jinan, China, in 2023, and the M.Sc. degree in applied statistics from China University of Petroleum, Qingdao, China, in 2026.

Her scientific interests include machine learning and swarm intelligence optimization.



Pu Tian received the B.Sc. degree in control engineering from Linyi University, Linyi, China, in 2021, and the M.Sc. degree in control engineering from the University of Jinan, Jinan, China, in 2024. She is currently pursuing the Ph.D. degree in applied mathematics and energy data science with China University of Petroleum, Qingdao, China.

Her scientific interests include machine learning and nonconvex optimization.



Ling Jian (Senior Member, IEEE) received the B.Sc. degree in applied mathematics from Qufu Normal University, Qufu, China, in 2003, the M.Sc. degree in operational research and cybernetics from Zhejiang University, Hangzhou, China, in 2006, and the Ph.D. degree in operational research and cybernetics from Dalian University of Technology, Dalian, China, in 2012.

Since March 2006, he has been with China University of Petroleum, Qingdao, China, where he is currently a Professor. His scientific interests include machine learning, data mining, and optimization.